

Phase 1 Report — Cityscapes Baseline Segmentation

Starting point

The original notebook (DeepLabV3+ / ResNet34, low-res squished 512×512 resize, per-batch mIoU, 10 epochs, no augmentation) produced **18.73% mIoU** — far below what an ImageNet-pretrained model should achieve. Diagnosis: the squished resize destroyed thin classes (poles, signs, people), there was no augmentation, the metric was a noisy per-batch average rather than the standard global confusion-matrix mIoU, and the LR schedule/epoch count was too weak.

Architecture and Training Setup Redesign

Rebuilt the training setup with the standard Cityscapes practices (crop-based augmentation, CE+Dice loss, AdamW+poly LR, global confusion-matrix mIoU). Then, targeting "minimum epochs, maximum mIoU," swapped the backbone entirely:

- **DeepLabV3+/ResNet34** → **SegFormer/MiT-B4** (ImageNet-pretrained) — converges faster and higher, and matches the encoder family DAFormer/MiC use later in the project.
- Added **OHEM-CrossEntropy + Dice loss**, AdamW at SegFormer's standard fine-tuning LR (6e-5) with linear warmup + poly decay, and **multi-scale + flip TTA** at final evaluation.

First real run — data ceiling discovered

Running on the Kaggle shuvoalok/cityscapes mirror crashed (`CropSizeError`) — investigation revealed this mirror is only **96×256** resolution, nowhere near real Cityscapes (2048×1024). Fixed the crash (adaptive crop sizing + padding safety net) and redesigned the augmentation pipeline for tiny images. The run completed cleanly but landed at only **21.8% mIoU** — correctly diagnosed as a **data resolution ceiling**, not a training bug (thin classes like poles/signs/riders become unrecoverable at that resolution).

Sourcing real data

Checked a reference model shared by the supervisor (Koushim/deeplabv3-resnet50-cityscapes) — found it was only 1-epoch trained and reported pixel accuracy (not mIoU), so not useful as a starting checkpoint; kept the SegFormer plan. Identified the **official cityscapes-dataset.com** as the reliable full-resolution source. Registered, downloaded `leftImg8bit_trainvaltest.zip` + `gtFine_trainvaltest.zip`, and verified locally: true **2048×1024**, 2975 train / 500 val correctly paired images↔labels.

Rewriting the pipeline for real data

Rewrote the dataset-loading cells to auto-detect the official `leftImg8bit/gtFine` folder structure (nested by city) and re-enabled proper crop-based augmentation, now that real full resolution was available.

A mid-training freeze, and a permanent fix

After uploading to Kaggle and starting the 28-epoch run, training silently froze at epoch 9 — diagnosed as a known **OpenCV-thread vs. PyTorch-DataLoader-worker deadlock**. Fixed with `cv2.setNumThreads(0)` and reduced worker count, and — since this cost lost progress once — added **automatic per-epoch checkpointing and resume**, so any future interruption only costs at most one epoch, not the whole run.

Final result: 73.18% mIoU (with TTA)

Restarted training ran cleanly through all 28 epochs, climbing steadily from 41.81% (epoch 1) to a best raw checkpoint of 72.21% (epoch 25), then +0.97% from TTA at final evaluation.

Per-class IoU shows the expected, healthy pattern for a correctly-trained model:

- **Strong:** road 97.4%, sky 93.3%, car 92.6%, vegetation 90.6%, building 90.4%
- **Weaker (expected — thin/rare classes):** pole 47.7%, fence 53.9%, rider 54.5%, wall 58.3%, traffic light 58.8%

This is a credible, literature-consistent baseline, ready to serve as the reference point for Phase 2 (ACDC domain-gap evaluation) and later DAFormer/MIC comparisons.

Results — Figures

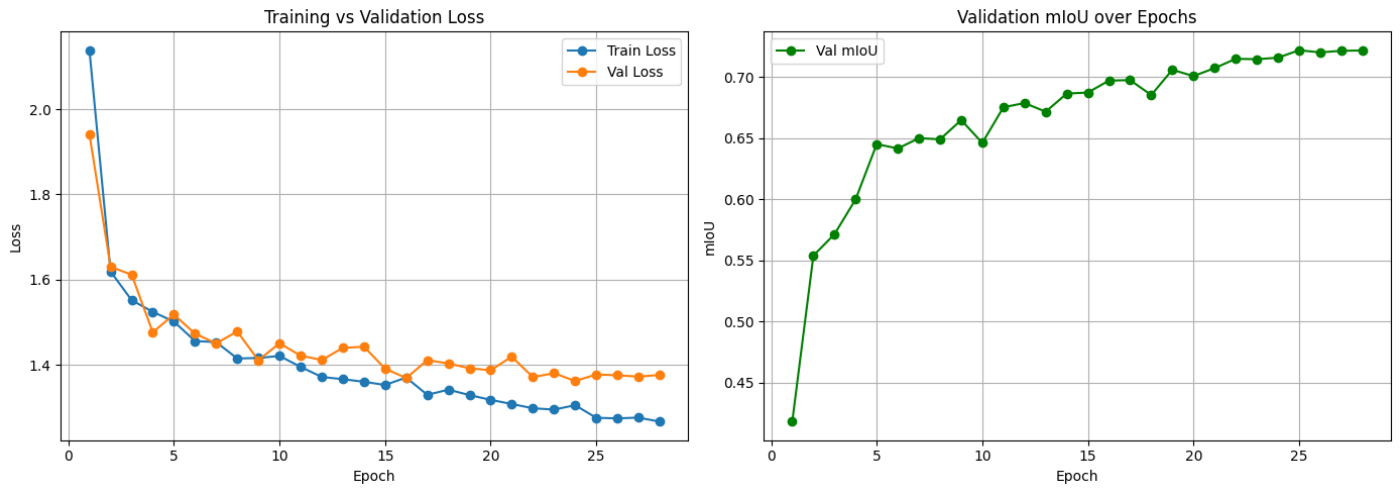


Figure 1: Training/validation loss and validation mIoU over 28 epochs (SegFormer/MiT-B4, full-resolution Cityscapes).

Phase 1 Baseline — Cityscapes Predictions

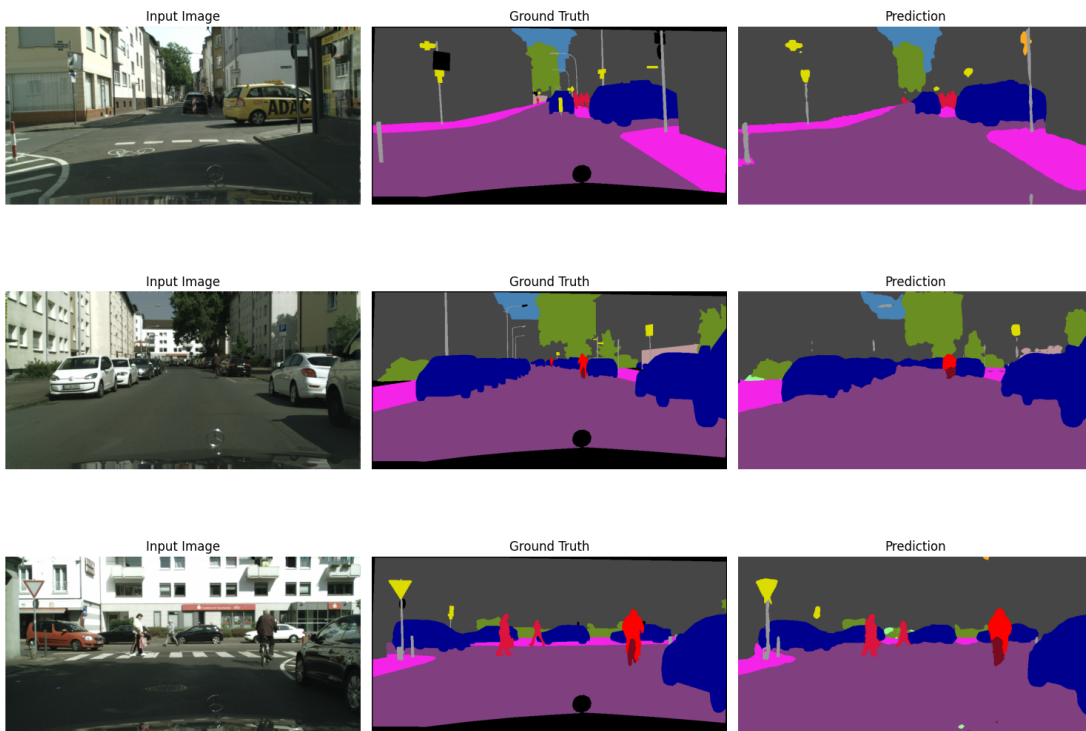


Figure 2: Qualitative predictions on Cityscapes validation images (input / ground truth / prediction).

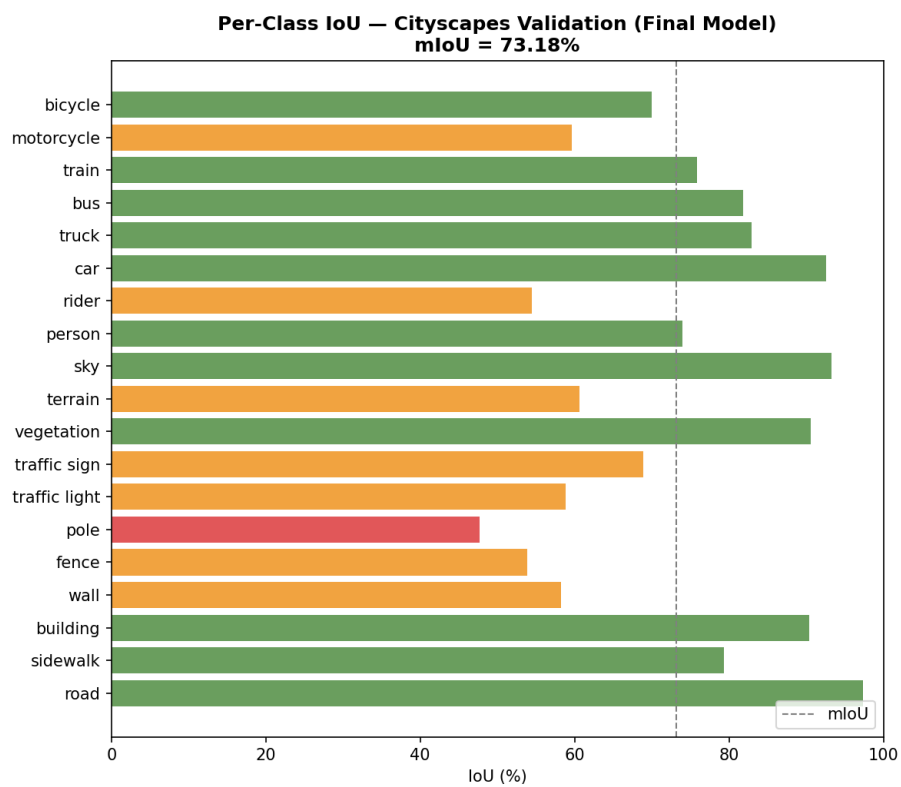


Figure 3: Per-class IoU for the final model (mIoU = 73.18% with TTA).